

AI Systems Performance Engineering

Chapter 6 — GPU Architecture, CUDA Programming, and Maximizing Occupancy Part III structure preview (draft)

Tianqin Cai
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

Part III — The Memory Ladder at a Glance

We can now create plenty of threads — but what are they all waiting for? Usually data.

Level	Scope	Capacity	Latency	BW
Registers	thread	256 KB / SM	~1 cyc	10s TB/s
Shared + L1	SM	228 KB usable	20–30 cyc	TB/s
TMEM	SM	256 KB (TC)	~10 cyc	TB/s
Const cache	SM	8 of 64 KB	1 cyc bcast	TB/s
L2 cache	GPU	126 MB	~200 cyc	multi-TB/s
Local (spill)	thread	DRAM-backed	≤1000 cyc	—
Global HBM3e	device	180 GB (B200)	≤1000 cyc	~8 TB/s

Table 6-5. Every level trades capacity for latency/bandwidth.

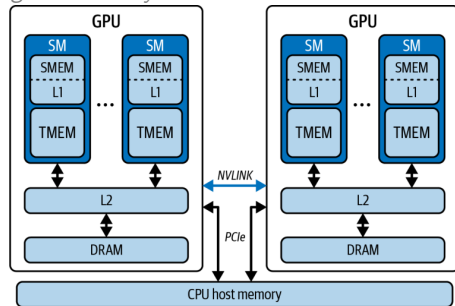


Figure 6-10. GPU memory hierarchy, including the CPU

*Plain English: your **own toolbox** (registers) → the **shared workbench** (shared) → the factory's **central depot** (L2) → the **distant warehouse** (HBM) — do the work at your bench.*

Register Pressure Can Turn Fast Storage into DRAM Access

- Every thread “begins its journey” at the **register file**: single-cycle reads/writes, contending with almost nothing — **tens of TB/s per SM**.
- Budget: 64K registers per SM, **max 255 per thread**.
- The cliff: need more (too many locals, compiler temporaries) $\Rightarrow$ overflow **spills into local memory** — which despite the name lives in **off-chip DRAM**: hundreds to 1000+ cycles.
- Watch **“Registers Per Thread”** in Nsight Compute — spills are silent killers.

Plain English: “local memory” is a self-storage unit across town, not your pocket.

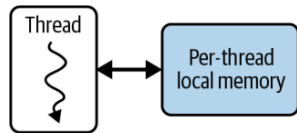


Figure 6-12. Per-thread local memory (spill space, DRAM-backed)

Shared Memory + L1: One SRAM, Two Jobs

- One **256 KB SRAM per SM**, two jobs:
hardware-managed **L1 cache** +
programmer-managed **shared memory** (up to 228 KB) — the workbench where **one block's threads** share data at $\sim 20\text{--}30$ cycles, **TB/s-class**.

Why it speeds things up: load once, reuse many

Matrix multiply: one block computes one **output tile**.

Without shared memory, threads 0, 1, 2, ... each **re-read the same slice of A** from global memory. With it:

Global $\rightarrow$ loaded **once** $\rightarrow$ Shared $\rightarrow$ reused by **every thread in the block**

- Two costs: use too much $\Rightarrow$ **fewer resident blocks** (occupancy); bad access layout $\Rightarrow$ **bank conflicts** (serialized access).

*Plain English: one workbench, adjustable split: your team's **project table** vs. the automatic **cache shelf**.*

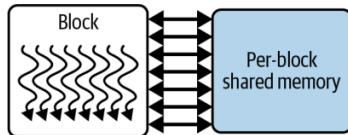


Figure 6-13. Thread-block shared memory

TMEM and TMA: Feeding the Tensor Cores *

* *Blackwell's new hardware — Chapter 10 covers it in depth; a 20-second flyover today.*

- **TMEM**: dedicated **256 KB per-SM SRAM**, the **accumulator** for 5th-gen Tensor Core ops (`tcgen05.*`, **UMMA**); talks to Tensor Cores at tens of TB/s.
- **Not pointer-addressable** from CUDA C++ — data movement is orchestrated by the **TMA** (Tensor Memory Accelerator) via **descriptors**.
- For $C = A \times B$: B in SMEM, A + accumulator in TMEM; **TMA** moves tiles HBM $\rightarrow$ L2 $\rightarrow$ SMEM (the TMEM side is implicit).
- Net effect: **reduces Tensor Core reliance on global memory**.

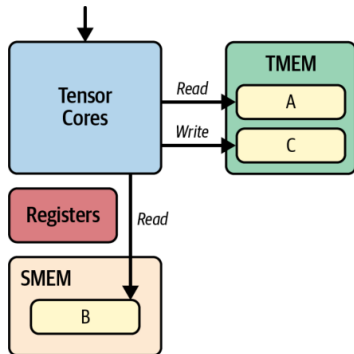


Figure 6-11. TMEM + SMEM servicing Tensor Cores for $C = A \times B$

Plain English: TMA is a forklift crew moving pallets in the background; the chefs never leave the kitchen.

Constant Cache: One-Cycle Broadcast

- **Constant memory** = a small, **read-only** region of global memory (64 KB `__constant__`); each SM fronts it with an **~8 KB constant cache**.
- **Superpower**: when all 32 threads of a warp read the **same address**, one fetch is **broadcast to all 32 lanes** — as fast as a register.
- **Weakness**: if the 32 threads read **different addresses**, the requests are **batched or even serialized**.

Good fit

Small, read-only, **uniformly accessed** parameters: RoPE tables, ALiBi slopes, LayerNorm γ/β , quantization scales.

Bad fit

Ordinary **large tensors**; tables where **each thread looks up a different entry**.

Plain English: a megaphone, not a mailbox: one announcement reaches all 32 workers at once — but only if they all asked the same question.

L2 Cache: The GPU-Wide Middleman

- **126 MB** shared by all SMs — the glue between on-chip SRAM and off-chip HBM3e.
- ~200 cycles, aggregate bandwidth in the **tens of TB/s**; absorbs L1 spillover.
- **Cross-block reuse**: data fetched by one thread block is reused by others **without revisiting DRAM**.

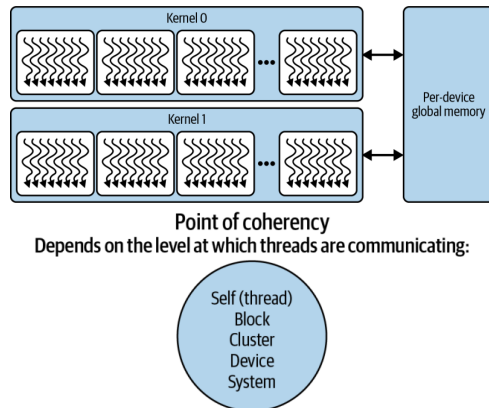
*Plain English: L2 is the factory's **central depot**: if another crew already fetched the part from the warehouse, you grab it here — provided everyone orders **whole pallets**, not single screws.*

The coalescing rule (book tip)

Structure global loads into **128-byte aligned, coalesced** segments that map cleanly to cache lines — avoids split transactions, maximizes L2 **and** DRAM bandwidth. (How-to: Chapter 7.)

Global HBM3e, the Dual-Die B200, and Coherency

- **180 GB** on B200 (~ 288 GB on B300) at ~ 8 **TB/s** — but **hundreds to 1000+ cycles**: the slowest link in the chain.
- Register spills and oversized automatic arrays (local memory) also pay this price.
- **Fun fact (book note)**: B200 = **two reticle-limited dies** joined by a **10 TB/s** chip-to-chip interconnect, 4 HBM3e stacks each — presented to you as **one GPU, one address space**.
- **Point of coherency** (Figure 6-15): memory consistency is established at thread $\rightarrow$ block $\rightarrow$ cluster $\rightarrow$ device $\rightarrow$ system scope — the wider the communication, the higher the cost.



Figures 6-14 / 6-15. Global memory; points of coherency

Unified Memory Hides Copies, Not Their Cost

- `cudaMallocManaged()`: one coherent CPU+GPU address space — no separate buffers, no manual `cudaMemcpy`.
- Under the hood: pages **migrate on demand** over whatever link connects CPU and GPU.
- The catch: a GPU touch of a CPU-resident page **page-faults and stalls the kernel**.
- On PCIe: on-fault transfers can be **slower than manual memcpy**. On Grace superchips, **NVLink-C2C** (~900 GB/s) makes migrations near device-native — **but latency is never zero**.

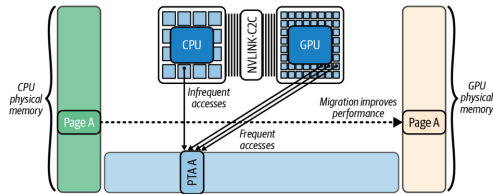


Figure 6-16. Automatic page migration with Unified Memory

*Plain English: Unified Memory is **automatic parts delivery**: convenient — but if the part isn't in the workshop when the crew needs it, **the crew stands and waits**.*

Taming Unified Memory: Prefetch, Advise, Attach

```
// 1) move data BEFORE the kernel needs it:
cudaMemPrefetchAsync(ptr, size, gpuId, s);
// 2) tell the driver how you'll use it:
cudaMemAdvise(ptr, size,
    cudaMemAdviseSetPreferredLocation, gpuId);
cudaMemAdvise(ptr, size,
    cudaMemAdviseSetReadMostly, gpuId);
cudaMemAdvise(ptr, size,
    cudaMemAdviseSetAccessedBy, otherGpuId);
// 3) confine faults/migrations to one stream:
cudaStreamAttachMemAsync(stream, ptr, 0,
    cudaMemAttachSingle);
```

- **Prefetch** turns first-touch migrations into **overlappable async transfers** (Figure 6-17).
- **ReadMostly** allows replicated read-only copies; **AccessedBy** lets a second GPU map pages **without** migration (Figure 6-18).
- **Attach** stops *other* streams from stalling on this range.
- No NVLink-C2C? Pin data **NUMA-local** (peer copy / prefetch).
- Result: performance **close to manual** `cudaMemcpy`, simplicity retained.